SpendWise — Personal Budget & Expense Tracker

Stack & Scale Remote Internship 2026 | Week 02 — Mobile Engineering

Deliverable: Monday (Day 1) Progress Package
Date: November 02, 2026
Author: Intern / Mobile Developer
DAY 1 FOUNDATION COMPLETE

FLUTTER ANALYZE	TEST SUITE	TARGET CURRENCY	STORAGE ARCHITECTURE
0 Issues (Passed)	**6 / 6 Passed**	**PKR (Minor Units)**	**Offline-First SQLite**

> **Data Separation Notice:** In strict alignment with Week 02 requirements, Day 1 features a complete UI, navigation, domain modeling, and initialized SQLite database schema. Figures shown on the Dashboard and History timeline are clearly labeled **Sample Presentation Data** and are not falsely presented as persisted records. Full transactional CRUD connects in Day 2.

1. Project Title & Internship Details

SpendWise is a modern, offline-first personal finance and expense tracking application engineered for students and early-career professionals. Built as the Week 02 project for the **Stack & Scale Remote Internship 2026**, SpendWise empowers users to monitor their income, track daily expenditures, set monthly category spending caps, and visualize savings progress without cloud dependencies, paid APIs, or third-party bank logins.

2. Monday's Objectives & Completed Deliverables

- **Complete 4-Screen Foundation:** Engineered polished interfaces for Dashboard, Add Transaction, Transaction History, and Budgets, along with a functional Settings & Diagnostics screen.
- **Full Bottom Navigation:** Implemented a 5-destination bottom navigation bar with active state pills and persistent tab state via `IndexedStack`.
- **Fintech Minor-Units Standard:** Implemented `TransactionModel` and `CategoryModel` with amounts stored strictly as integers in minor currency units (paisas) to eliminate floating-point rounding errors.
- **Centralized PKR Currency Architecture:** Configured Pakistani Rupee (PKR / Rs.) with centralized formatting logic.
- **SQLite Local Storage Foundation:** Implemented `AppDatabase` singleton, created tables (`categories`, `transactions`, `budgets`), enforced SQLite foreign keys via PRAGMA, created index optimizations, and pre-seeded 11 default categories.
- **Desktop & Mobile Support:** Integrated `sqflite_common_ffi` allowing tests and desktop runners to execute alongside native Android.
- **Rigorous Verification:** 100% clean `flutter analyze` check and 100% passing automated test suite (6/6 tests).

3. Overview of Premium Fintech UI Design

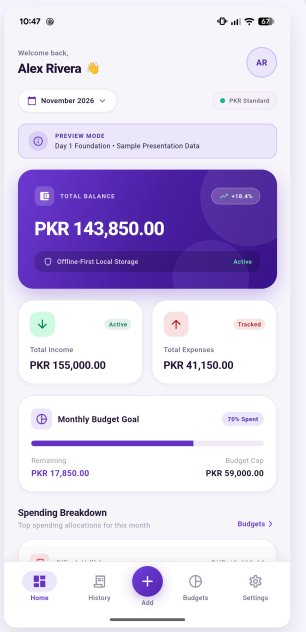
SpendWise was designed from the ground up to match the visual fidelity of tier-one fintech applications. The visual direction features:

- **Brand Identity:** Distinctive violet/purple brand identity (`#6536C7`) complemented by deep purple accents (`#4F28A4`) and lavender surfaces (`#F8F6FE`, `#F6F4FC`).

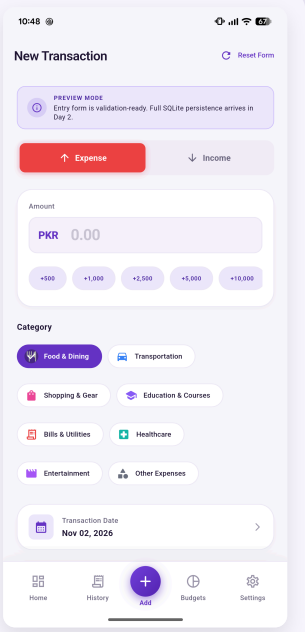
- **Semantic Color Hierarchy:** Emerald Green (`#10B981`) for positive cash flow / income, Coral Red (`#EF4444`) for expenditures, and Amber (`#F59E0B`) for budget threshold warnings.
- **Design System:** Custom rounded cards (22px radius), subtle multi-layered drop shadows, Material 3 typography scale, and responsive layout constraints (avoiding hardcoded pixel overflow).

4. Evidence: Genuine Screenshots from Physical Device (Pixel 6 Pro)

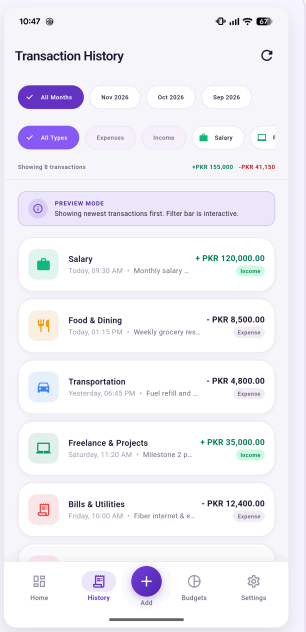
The screenshots below were captured directly from the running application on an Android device (Pixel 6 Pro, Android 17):



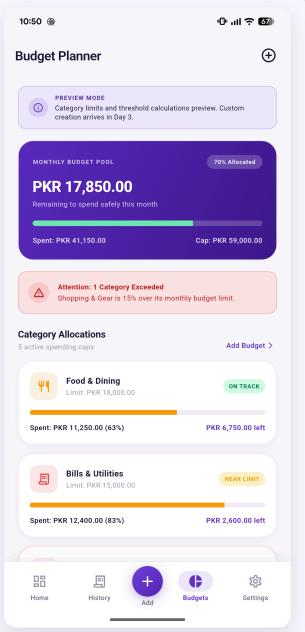
A. Dashboard
Balance & Metrics



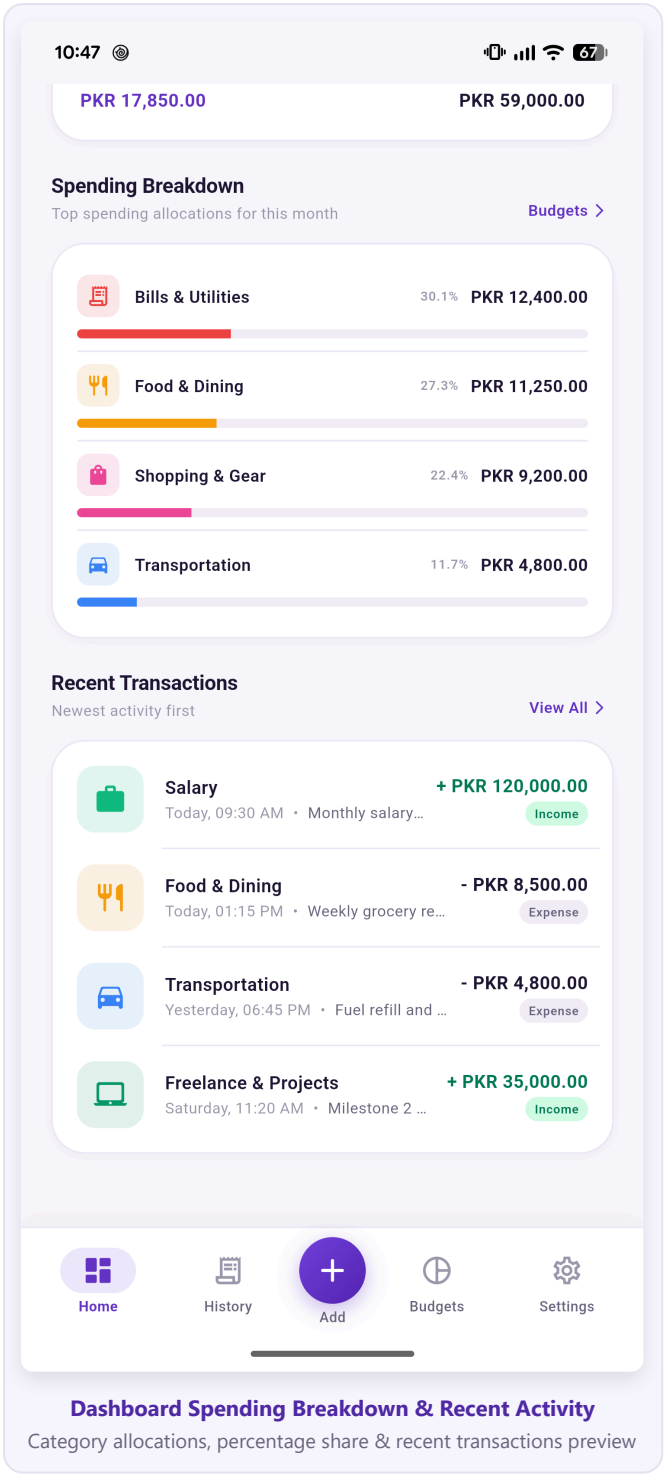
B. Add Transaction
Validation Form



C. History
Timeline & Filters

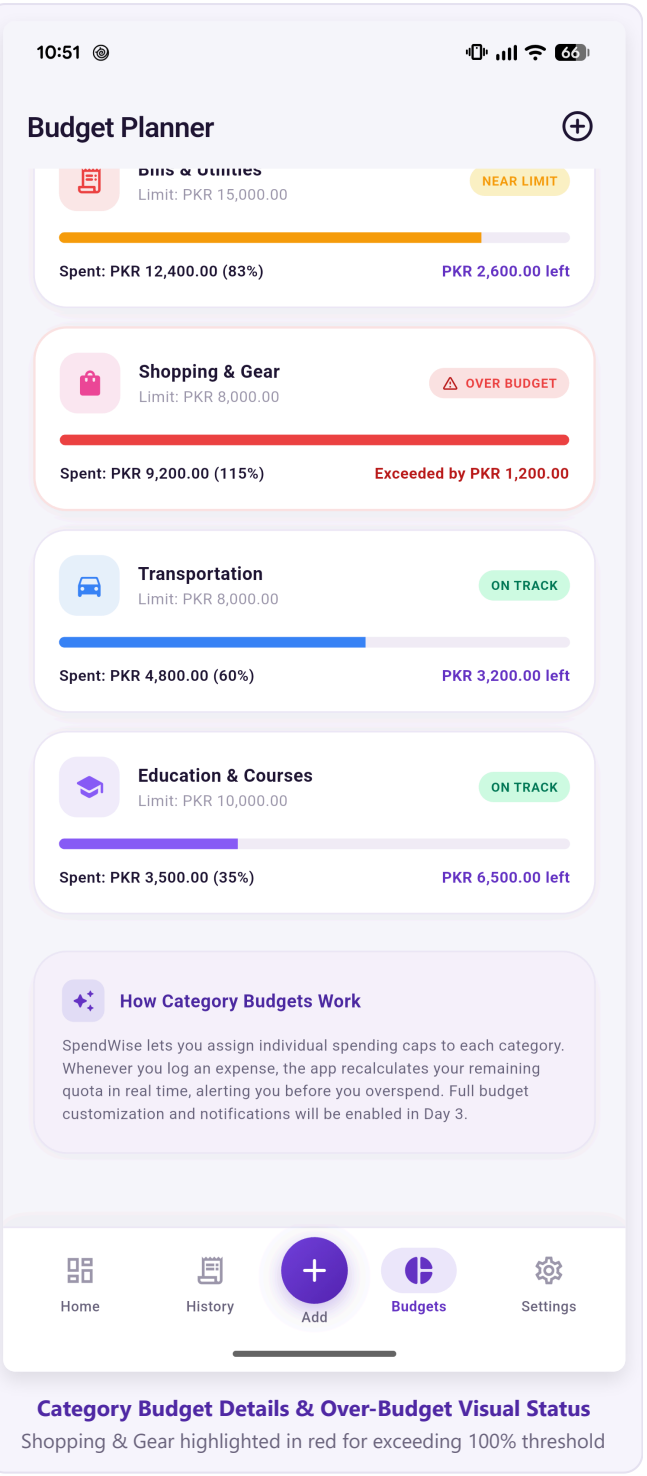


D. Budgets
Caps & Alerts



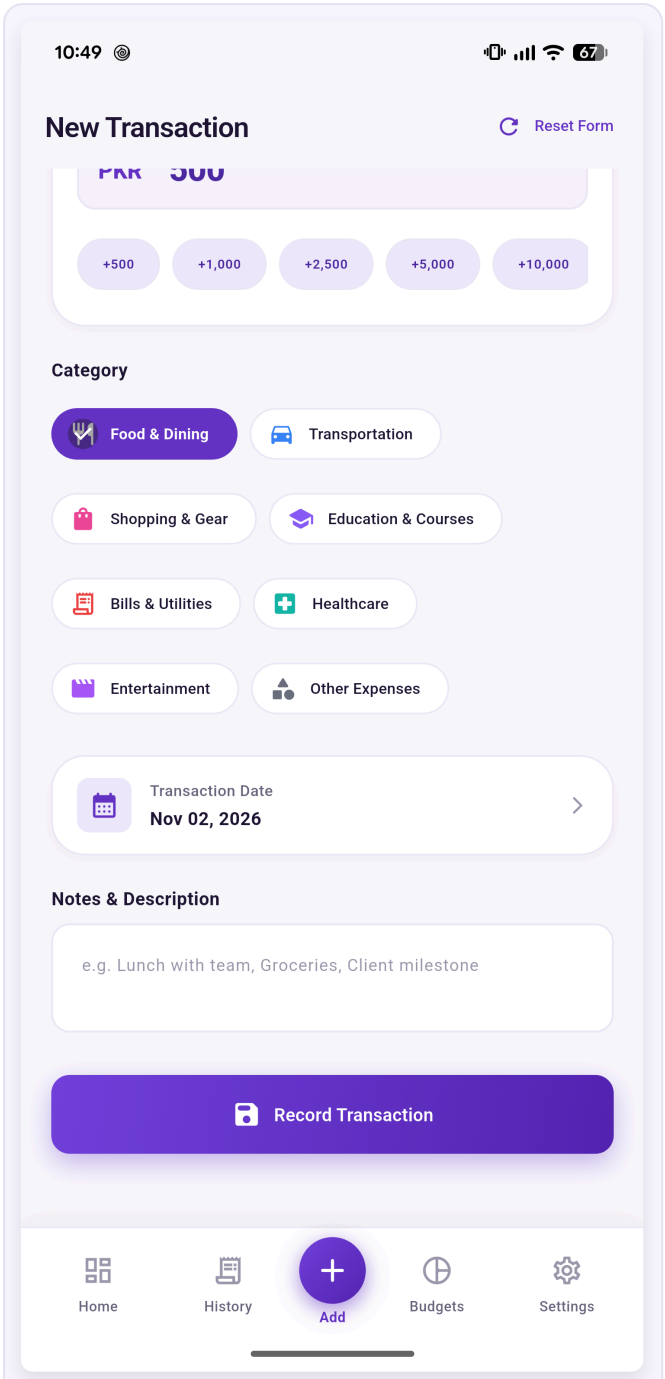
Dashboard Spending Breakdown & Recent Activity

Category allocations, percentage share & recent transactions preview



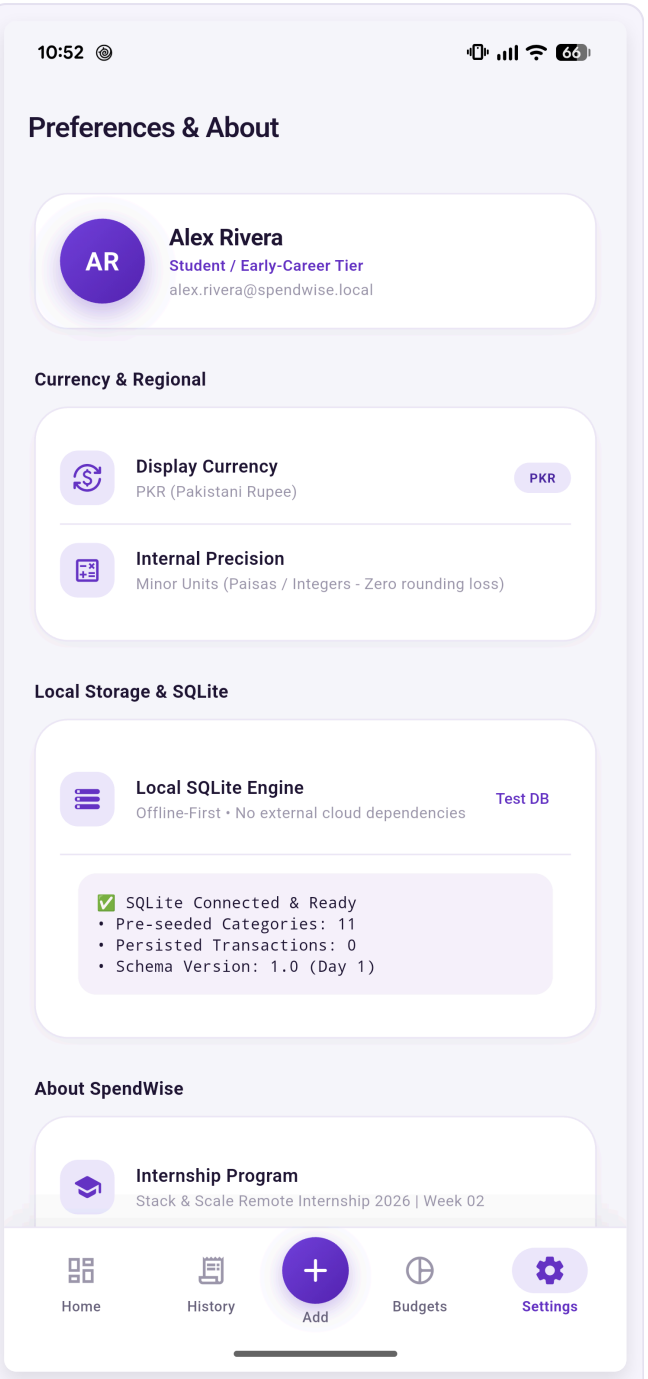
Category Budget Details & Over-Budget Visual Status

Shopping & Gear highlighted in red for exceeding 100% threshold



Add Transaction Form Validation & Feedback

Clean dialog validating inputs and confirming Day 2 persistence readiness



Settings & SQLite Database Diagnostic Test

Live query verifying SQLite connection, schema v1.0, and 11 seeded categories

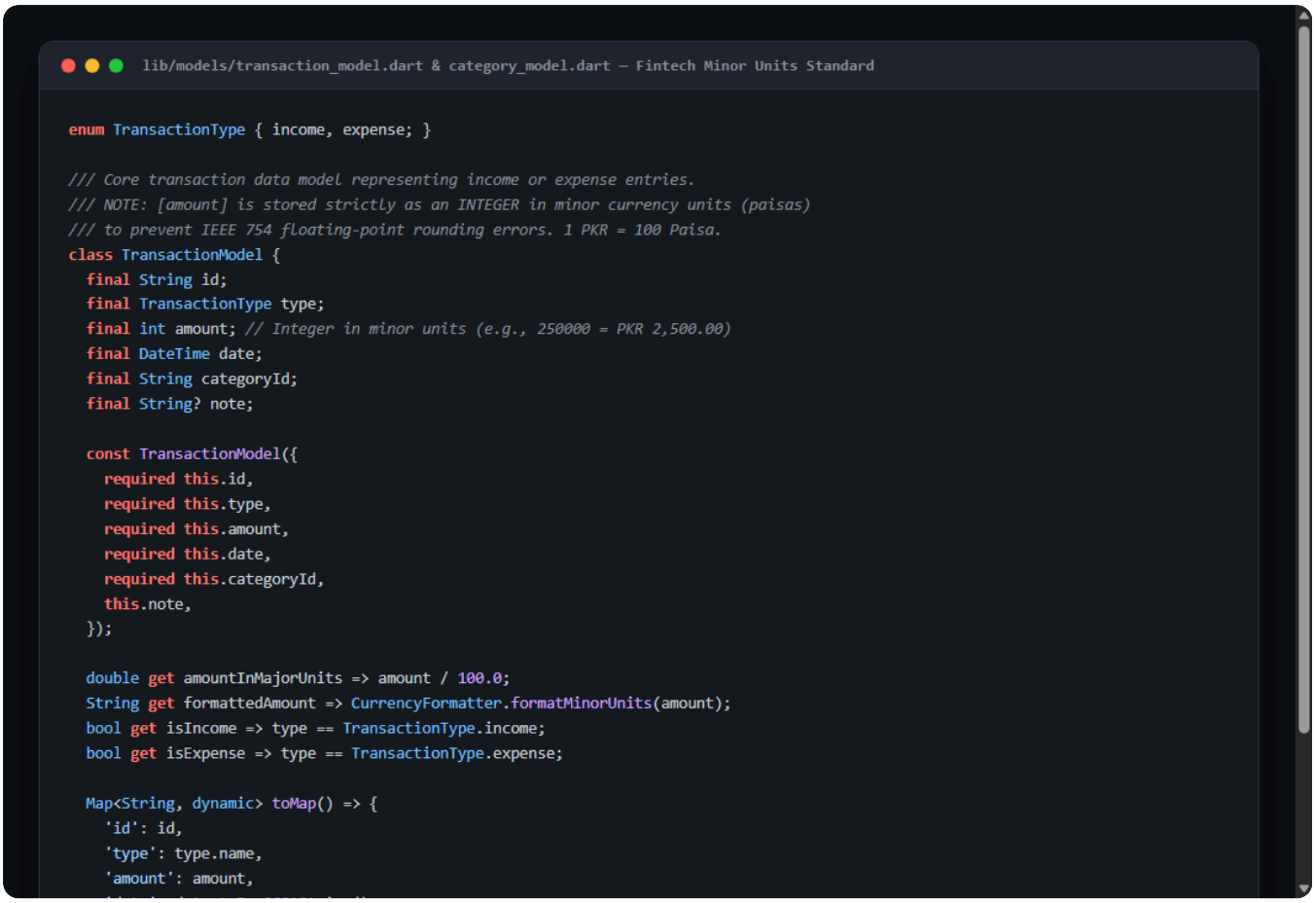
5. Flutter Project Architecture & Folder Structure

The code is strictly separated into modular, maintainable directories:

Directory	Responsibility
`lib/app/`	Application root, Material 3 theme configuration, typography, and bottom navigation scaffold.
`lib/core/constants/`	Fintech color palette (`AppColors`), dimension tokens, and string constants.
`lib/core/database/`	SQLite connection singleton (`AppDatabase`), schema DDL, indexes, and repository helper (`DatabaseService`).
`lib/core/utils/`	Centralized PKR currency formatter and relative date formatting logic.
`lib/core/widgets/`	Reusable UI components: `CustomCard`, `MetricCard`, `PreviewBanner`, `PrimaryButton`, `SectionHeader`.
`lib/models/`	Immutable domain models with minor-unit integer precision and serialization.
`lib/data/`	Explicitly identified sample presentation fixtures for Day 1 preview.
`lib/features/`	Feature modules containing screens and specialized widgets for Dashboard, Transactions, History, Budgets, and Settings.
`test/`	Automated unit tests for models/SQLite and widget integration tests for navigation.

6. Transaction & Category Data Models (Code Evidence)

Fintech standard: All monetary figures are stored as integers in minor currency units (paisas / cents). 1 PKR = 100 Paisa.



7. SQLite Database Setup & Schema (Code Evidence)

Foreign keys are enforced via `PRAGMA foreign_keys = ON;`, indexes optimize queries by date and category, and default categories are seeded inside a single batch transaction on database creation:

lib/core/database/app_database.dart — Local SQLite Singleton, Foreign Keys & Pre-Seeding

```dart
class AppDatabase {
  static final AppDatabase instance = AppDatabase._internal();

  Future<Database> get database async {
    if (_database != null && _database!.isOpen) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<void> _onConfigure(Database db) async {
    // Enforce referential integrity on every SQLite connection
    await db.execute('PRAGMA foreign_keys = ON;');
  }

  Future<void> _onCreate(Database db, int version) async {
    final batch = db.batch();
    // 1. Create Tables: Categories, Transactions, Budgets
    batch.execute(DatabaseConstants.createCategoriesTable);
    batch.execute(DatabaseConstants.createTransactionsTable);
    batch.execute(DatabaseConstants.createBudgetsTable);

    // 2. Create High-Performance Indexes
    batch.execute(DatabaseConstants.createTransactionsDateIndex);
    batch.execute(DatabaseConstants.createTransactionsCategoryIndex);

    // 3. Batch Pre-Seed 11 Default Categories into SQLite
    _seedDefaultCategories(batch);
    await batch.commit(noResult: true);
  }
}
```

8. Working Navigation Architecture (Code Evidence)

SpendWise features a 5-destination bottom navigation bar powered by `IndexedStack` to preserve user input and scroll positions across tabs:

lib/app/navigation/app_scaffold.dart — 5-Destination IndexedStack & Custom Bottom Bar

```dart
class AppScaffold extends StatefulWidget { ... }

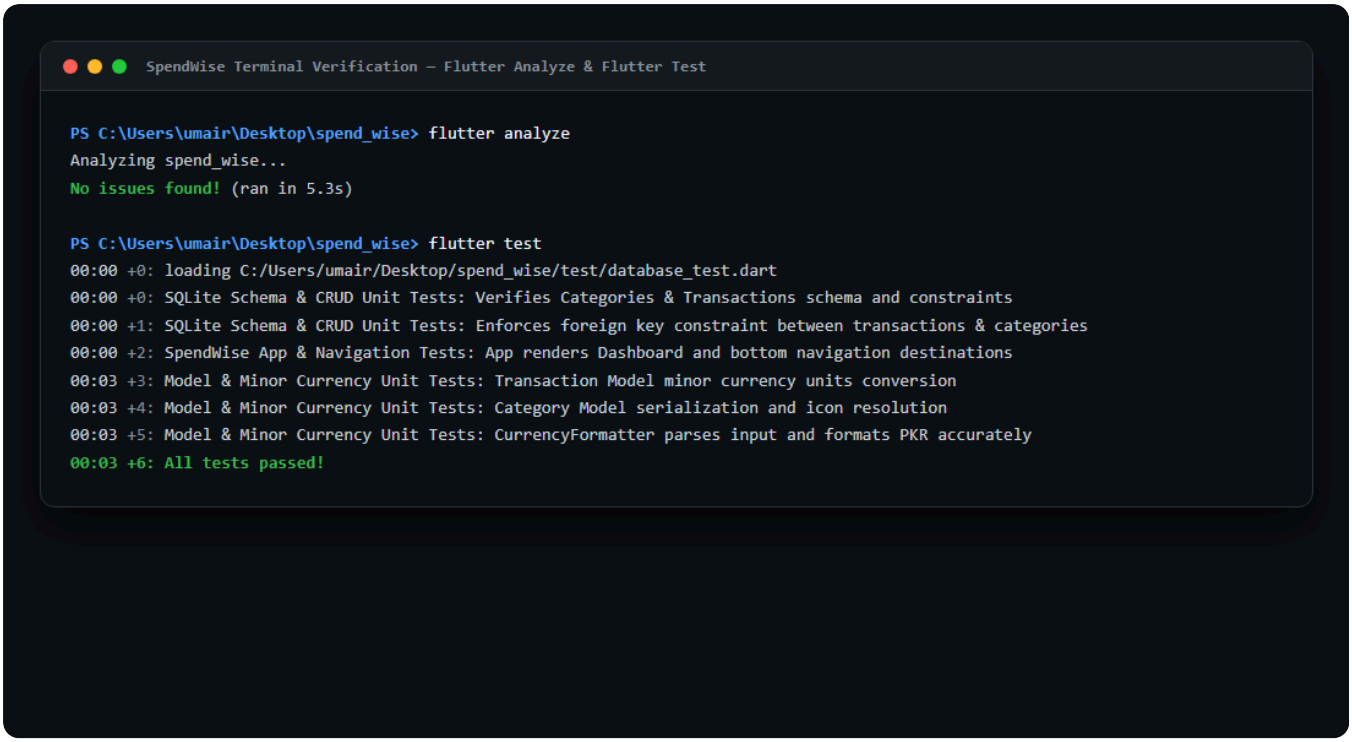
class _AppScaffoldState extends State<AppScaffold> {
  int _currentIndex = 0;

  @override
  Widget build(BuildContext context) {
    final screens = [
      DashboardScreen(
        onNavigateToHistory: () => _onTabSelected(1),
        onNavigateToAddTransaction: () => _onTabSelected(2),
        onNavigateToBudgets: () => _onTabSelected(3),
      ),
      HistoryScreen(onAddTransactionTap: () => _onTabSelected(2)),
      AddTransactionScreen(onTransactionSavedPreview: () => _onTabSelected(0)),
      const BudgetsScreen(),
      const SettingsScreen(),
    ];

    return Scaffold(
      // Preserves scroll offset & form state across all 5 destinations
      body: IndexedStack(index: _currentIndex, children: screens),
      bottomNavigationBar: _buildBottomNavigationBar(),
    );
  }
}
```

9. Verification Evidence: Flutter Analyze & Automated Tests

All automated checks pass with zero warnings, zero errors, and six comprehensive tests:



10. Challenges Encountered & Solutions

- **Challenge 1 (Host Disk Space Exhaustion):** During the initial build, Windows reported a disk-full error (OS Error 112).
  Solution: Cleaned system temp caches and emptied the Recycle Bin, recovering over 2.5 GB of free space and unblocking the compiler.
- **Challenge 2 (Material 3 CardTheme Compatibility):** In modern Flutter (v3.47+), assigning a legacy `CardTheme` to `cardTheme` triggered a type mismatch lint.
  Solution: Updated the theme configuration to use `CardThemeData` in full alignment with the latest Flutter Material 3 SDK.
- **Challenge 3 (Desktop vs. Mobile SQLite Support):** Standard `sqflite` requires native Android/iOS channels and can fail when running standard Dart unit tests or Windows desktop builds.
  Solution: Integrated `sqflite_common_ffi` and implemented dynamic platform detection in `AppDatabase`, allowing desktop runners and in-memory unit tests to execute SQLite queries flawlessly.

11. What I Learned Today

- How real-world fintech applications prevent floating-point rounding errors by storing money as integer minor currency units (paisas) and converting only at the display boundary.
- How to design a cohesive, professional fintech design system in Flutter using Material 3 color schemes, custom surfaces, and consistent typographic hierarchy.
- The architecture of local-first Flutter apps: initializing SQLite safely, enforcing relational integrity with foreign keys, and batch pre-seeding categories.
- How to build testable, modular Flutter code using `IndexedStack` navigation and writing widget tests that verify multi-tab user flows.

12. Remaining Work Planned for Tuesday (Day 2)

- **Live Transaction CRUD:** Connect the `AddTransactionScreen` form directly to SQLite via `DatabaseService.insertTransaction()`.

- **Dynamic History Feed:** Replace sample history data with a reactive stream/FutureBuilder listening to real saved database transactions.
- **Transaction Deletion & Editing:** Implement swipe-to-delete and detail/edit dialogs for recorded expenses and income.
- **Dashboard Live Sync:** Bind the Dashboard total balance, monthly income, and monthly expense cards to real aggregate SQLite queries.